

An abstract graphic featuring a thick, flowing blue ribbon that curves from the top left towards the bottom right. The ribbon has a glossy, 3D appearance with highlights and shadows. Several light blue circles of varying sizes are scattered along the path of the ribbon. The background is white with faint, repeating watermarks of the word 'depositphotos' and a grid pattern.

El Proceso Unificado y el Ciclo de Vida del Software

Es una realidad...

**“Sólo el 29% de los
proyectos de software
tienen éxito.”**

Standish Group, CHAOS Report, 2015



Hechos del software

- Encontrar y corregir un error durante Producción cuesta 100 -1000 veces más que en el Diseño.
- El tiempo de desarrollo no puede comprimirse más de un 25% aumentando el número de gente.
- Por cada \$1 gastado en desarrollo, se gastan \$2 en mantenimiento.
- Los costos de desarrollo y mantenimiento están directamente en función de las actividades manuales.
- Programación representa solo un 15% del “esfuerzo” de desarrollo.
- En desarrollo de software existe diseconomía de escala (costo/línea)

“Industrial Software Metrics Top 10 List”

Barry Boehm



Síntomas de la crisis del Software

- ✗ No son cubiertas las necesidades del negocio.
- ✗ Requerimientos mal definidos.
- ✗ Módulos que no se integran.
- ✗ Difícil de mantener.
- ✗ Tardío descubrimiento de defectos.
- ✗ Baja calidad.
- ✗ Usuario finales insatisfechos.
- ✗ Bajo desempeño sobre altas cargas.
- ✗ Esfuerzo no coordinado del equipo.
- ✗ Liberaciones (Build-and-release issues)



Problemática del desarrollo de Software



Dirección

Maximizar
Beneficios



Jefe de Proyecto

Conducir
el equipo al
éxito



Desarrollador

Crear
Software
de
calidad



Problemática del desarrollo de Software



Dirección

- ◆ Retraso en proyectos
- ◆ Oportunidades perdidas
- ◆ Amenazas de la competencia



Jefe de Proyecto

- ◆ Fechas ajustadas
- ◆ Recortes de presupuestos
- ◆ Pocos recursos.



Desarrollador

- ◆ Reuniones
- ◆ Cambios
- ◆ Rehacer trabajo hecho

Maximizar
Beneficios

Conducir
el equipo al
éxito

Crear
Software
de
calidad



Problemática del desarrollo de Software



Dirección

- ◆ Retraso en proyectos
- ◆ Oportunidades perdidas
- ◆ Amenazas de la competencia



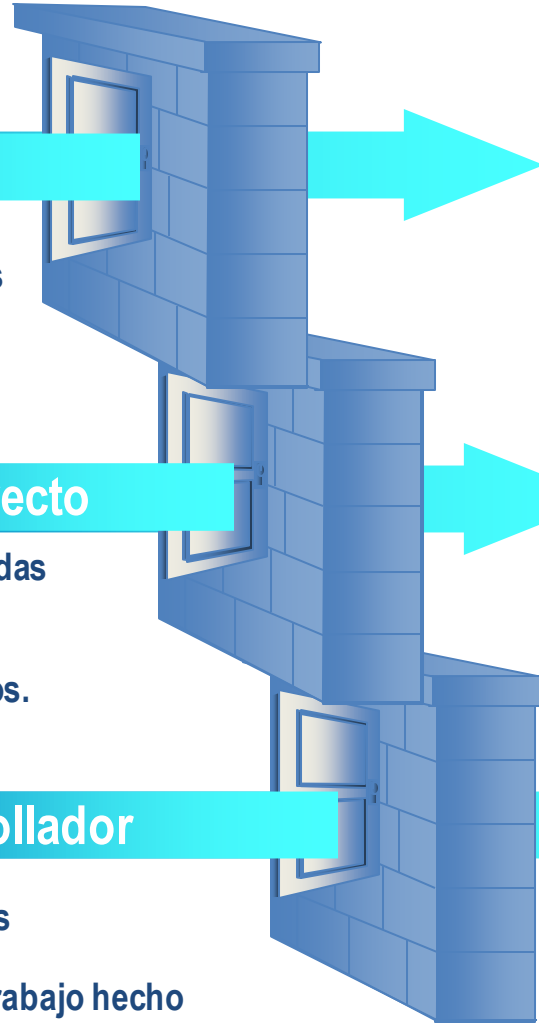
Jefe de Proyecto

- ◆ Fechas ajustadas
- ◆ Recortes de presupuestos
- ◆ Pocos recursos.



Desarrollador

- ◆ Reuniones
- ◆ Cambios
- ◆ Rehacer trabajo hecho



Maximizar
Beneficios

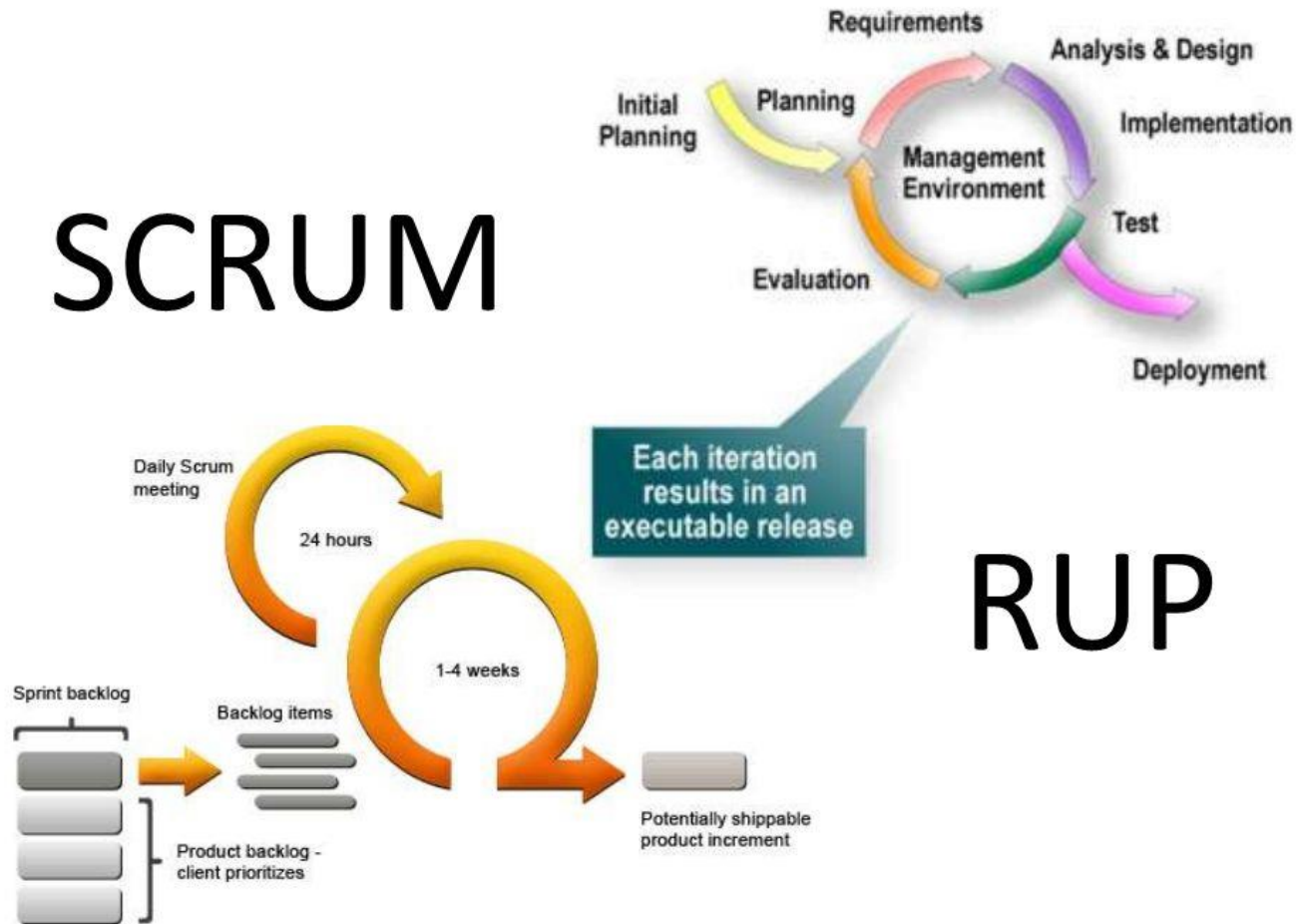
Conducir
el equipo al
éxito

Crear
Software
de
calidad



Metodologías de desarrollo de software

SCRUM



RUP



Metodología RUP

RUP es un proceso o marco de trabajo para el desarrollo de un proyecto de un software que define claramente quien, cómo, cuándo y qué debe hacerse en el proyecto



Ventajas

- ✓ Basada en mejores prácticas de metodologías
- ✓ Reduce el Riesgo de proyecto
- ✓ Proporciona una visión común del proceso de desarrollo
- ✓ Unifica a un equipo de software
- ✓ Incorpora fielmente el objetivo de la calidad

Desventajas

- ✓ Pretende prever y tener el control de antemano
- ✓ Modelo genera trabajo adicional
- ✓ Genera muchos costos
- ✓ No recomendable para proyectos pequeños



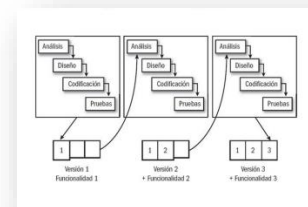
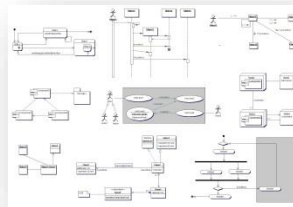
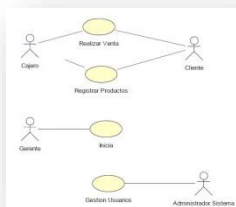
Metodología RUP

Basado en seis principios claves de desarrollo

1. Proceso adaptable
2. Equilibra prioridades
3. Demuestra valor iterativamente
4. Colaboración entre equipos
5. Eleva el nivel de abstracción
6. Se enfoca en la calidad

Se caracteriza por :

1. Basado Casos de Usos
2. Centrado en la Arquitectura
3. Modelo de ciclo de Vida es iterativo e incremental

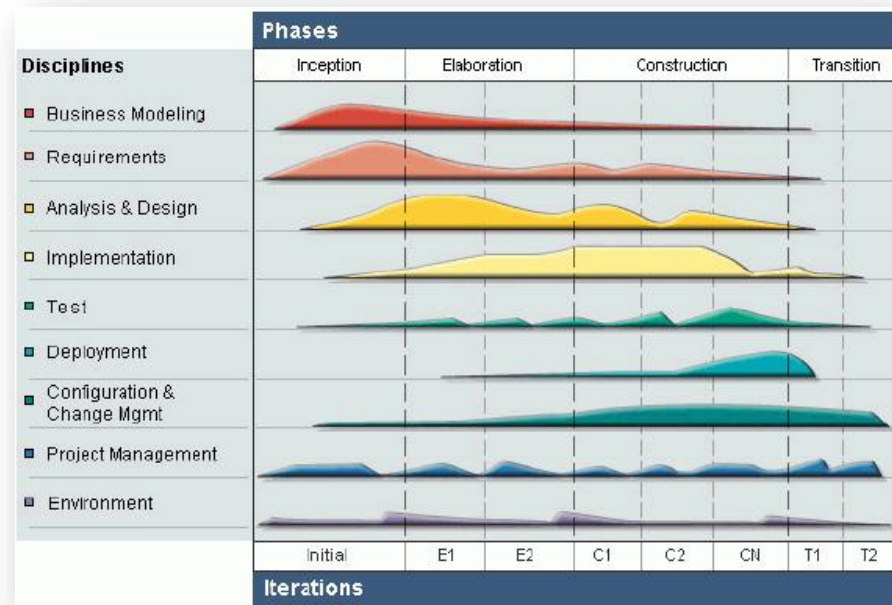


Metodología RUP

Basado en Mejores Practicas



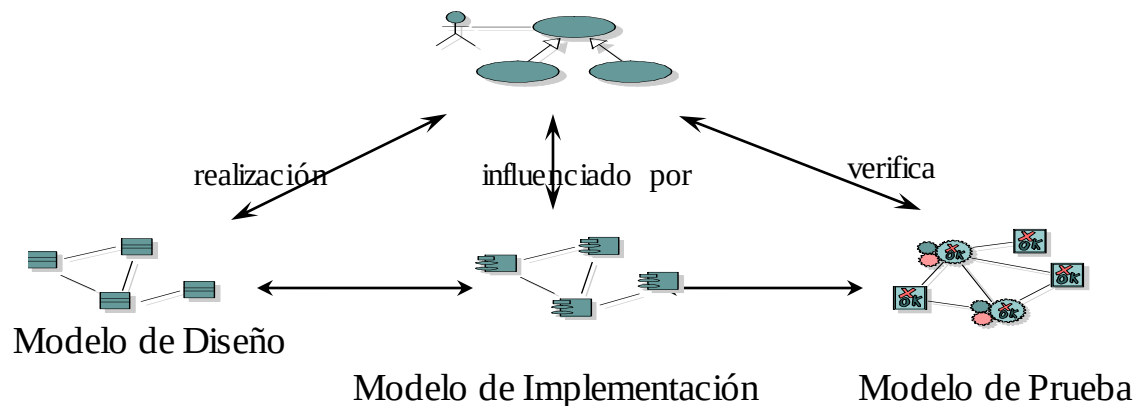
Estructura del RUP



Metodología RUP : Mejores Prácticas

1.- Administración de Requerimientos

- ❖ Licitar, organizar, y documentar la funcionalidad y restricciones requeridas.
- ❖ Llevar un registro y documentación de cambios y decisiones.
- ❖ Los requerimientos de negocio son fácilmente capturados y comunicados a través de casos de uso.
- ❖ Los casos de uso son instrumentos importantes de planeación.



Metodología RUP : Mejores Prácticas

2.- Desarrollo Iterativo del Software

- ❖ Dados los sistemas de software sofisticados de la actualidad, no es posible hacer de manera secuencial la definición completa del problema, diseñar la solución completa, construir el software y por último probarlo.
- ❖ El descubrimiento de defectos en fases posteriores de diseño dan como resultado un aumento en los costos y/ó la cancelación del proyecto.



*El tiempo y dinero gastados en la implementación de un diseño fallido, son **no recuperables***



3.- Arquitectura Basada en Componentes

- ❖ Se enfoca en el pronto desarrollo de una arquitectura ejecutable robusta.
- ❖ Resistente al cambio mediante el uso de interfaces bien definidas.
- ❖ Intuitivamente comprensible.
- ❖ Promueve un reuso más efectivo de software.
- ❖ Es derivada a partir de los casos de uso más importantes.



Metodología RUP : Mejores Prácticas

4.- Modelamiento Visual

- ❖ Captura la estructura y comportamiento de arquitecturas y componentes.
- ❖ Muestra como encajan de forma conjunta los elementos del sistema.
- ❖ Mantiene la consistencia entre un diseño y su implementación.
- ❖ Promueve una comunicación no ambigua.

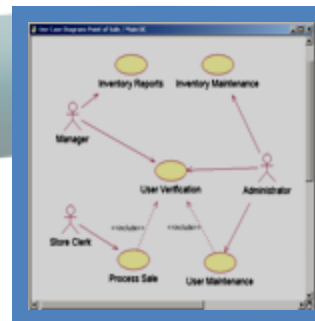
Modelado de Negocio



Modelado Web



Modelado de Requisitos



Modelado de Aplicaciones

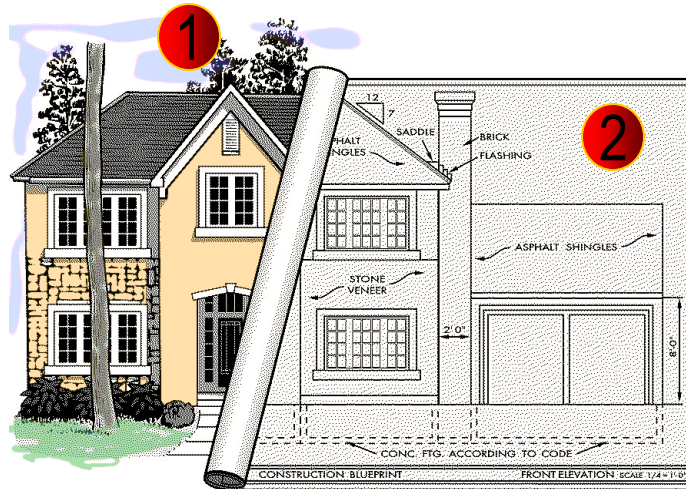


Modelado de Datos



Metodología RUP : Mejores Prácticas

4.- Modelamiento Visual



Perspectivas arquitectónicas de una casa



Metodología RUP : Mejores Prácticas

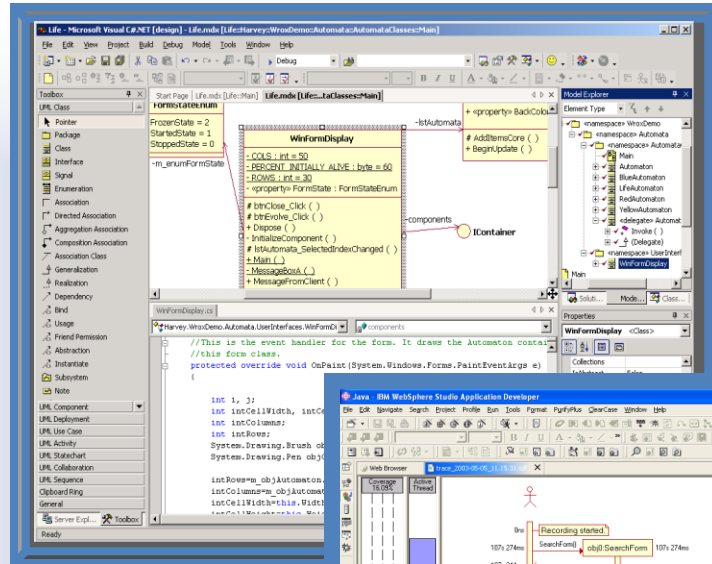
4.- Modelamiento Visual



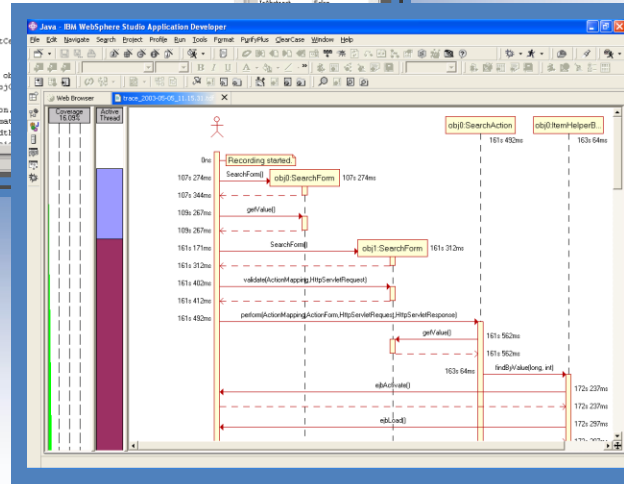
**Analista de
Negocio**



Arquitecto



Desarrollador



Diseñador DB

Una Herramienta para todo el equipo.



5.- Verificación de la Calidad

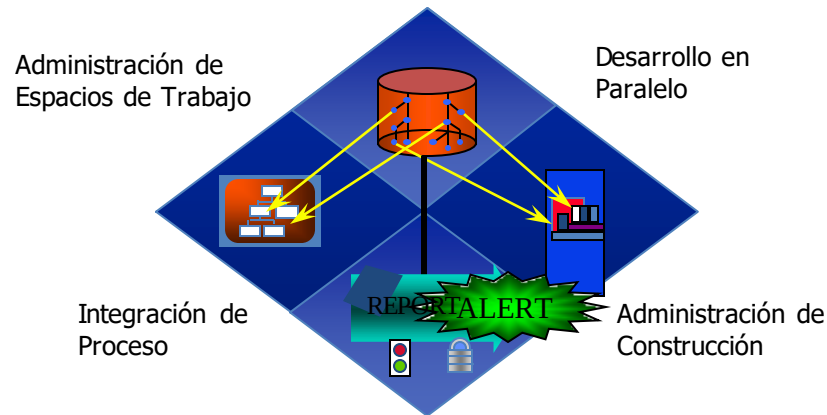
- ❖ Crea pruebas para cada escenario (casos de uso) para asegurar que todos los requerimientos están propiamente implementados.
- ❖ Verifica la calidad del software con respecto a los requerimientos basados en la confiabilidad, funcionalidad, desempeño de la aplicación y del sistema.
- ❖ Prueba cada iteración

Los problemas del software son de 100 a 1000 veces mas costosos de encontrar y reparar después del desarrollo



6.- Control de Cambios

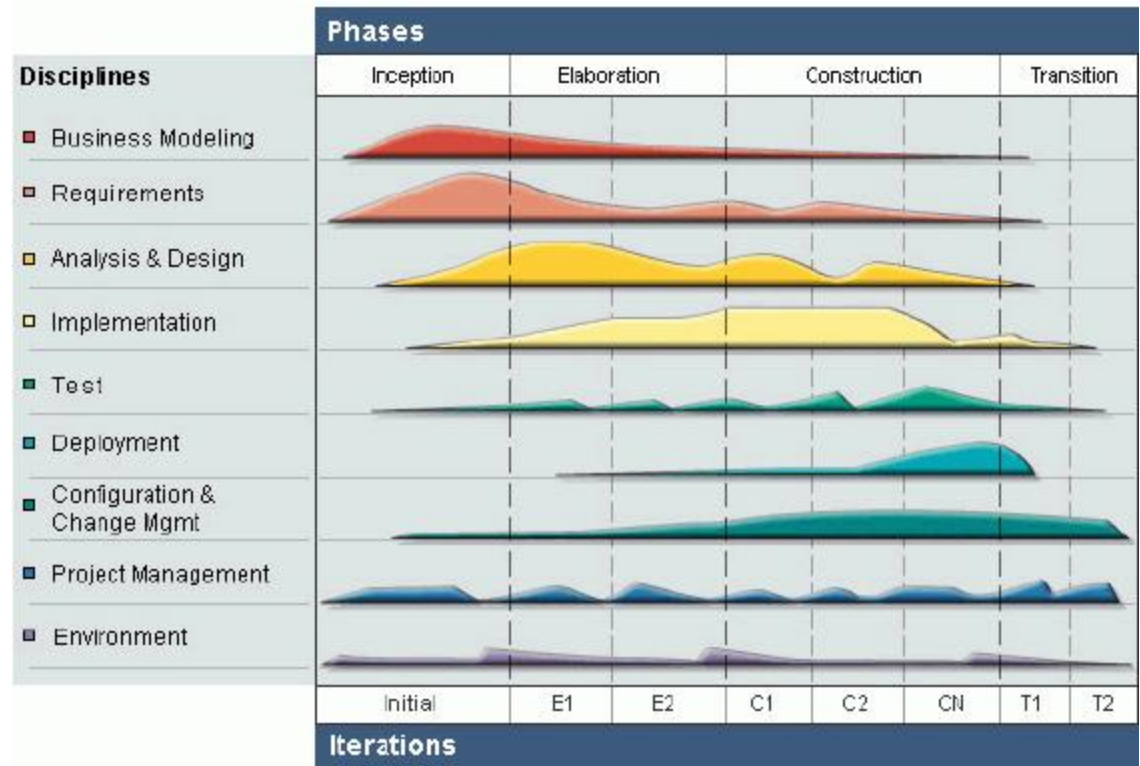
- ❖ Controlar, llevar un registro y monitorear cambios para permitir un desarrollo iterativo.
- ❖ Establece espacios de trabajo seguros para cada desarrollador
 - Provee aislamiento de cambios hechos en otros espacios de trabajo
 - Controla todos los artefactos de software – modelos, código, documentos, etc...



Metodología RUP : Estructura del RUP

Fases

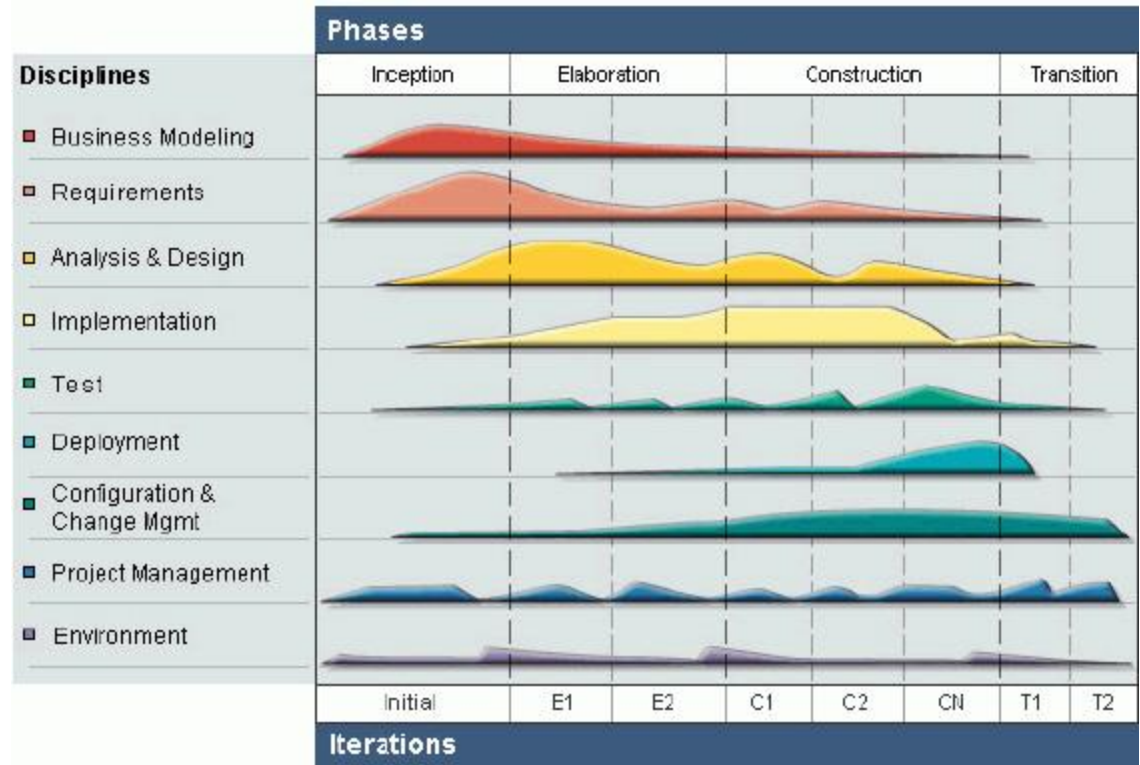
- **Inicio:** En esta primera fase se define el alcance y objetivos del proyecto.
- **Elaboración:** Se desarrolla el plan del proyecto, la especificación de características y la arquitectura base del sistema.
- **Construcción:** Esta fase se concentra en la elaboración, de un producto totalmente operativo y eficiente y el manual de usuario.
- **Transición:** Fase en el cual se instala el producto en el cliente y se entrena a los usuarios.



RUP : Estructura del RUP

Flujo de Trabajo del Proceso

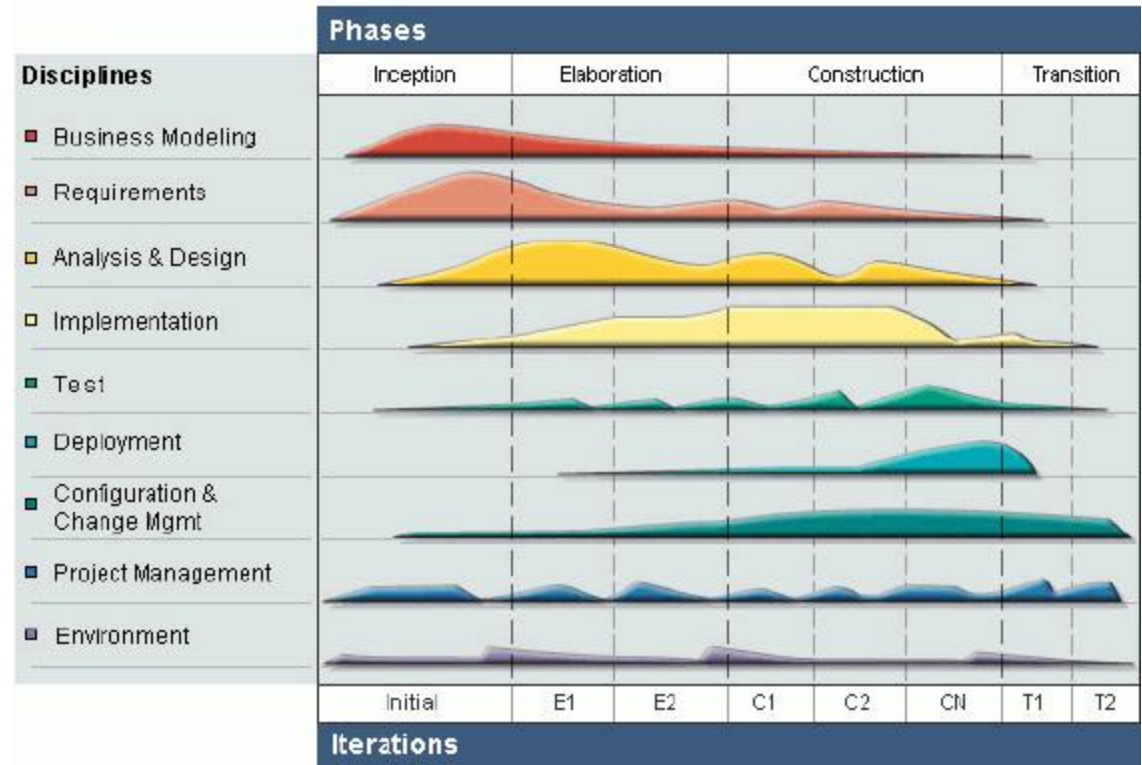
- **Modelado del negocio:** Describe la estructura y la dinámica de la organización donde se va a implantar el sistema que construyamos.
- **Requisitos:** Establece exactamente lo que tiene que hacer el sistema, para ello se extrae los requisitos utilizando diferentes métodos.
- **Análisis y Diseño:** Traduce los requisitos a una especificación que describe cómo implementar el sistema, creando para ello, diferentes vistas arquitectónicas.
- **Implementación:** Tiene en cuenta el desarrollo de software, las pruebas unitarias y la integración.
- **Pruebas:** Describe la ejecución de pruebas y las métricas para rastreo de defectos.
- **Despliegue o Implantación:** Incluye actividades relacionadas con la entrega de la aplicación.



RUP : Estructura del RUP

Flujo de Trabajo de Apoyo

- **Configuración y Control de cambios:** Mantiene la integridad de todos los artefactos que se crean en el proyecto. También mantiene información del proceso evolutivo que se ha seguido
- **Gestión del proyecto:** Es el arte de lograr un balance al gestionar objetivos, riesgos y restricciones para desarrollar un producto que sea acorde a los requisitos de los clientes y los usuarios.
- **Entorno:** Cubre la infraestructura necesaria para desarrollar un sistema.

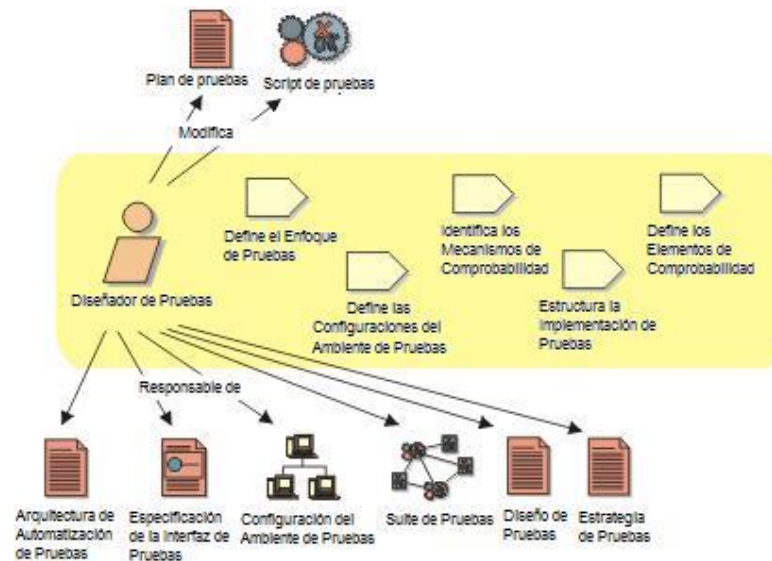


Roles en RUP

ROL

Define el comportamiento y responsabilidades de un individuo, o de un grupo de individuos trabajando juntos como un equipo.

Las responsabilidades de un rol son tanto llevar a cabo un conjunto de actividades como ser el dueño de un conjunto de artefactos



Roles en RUP

Analista

- Analista de Procesos de Negocio
- Diseñador de Negocio
- Analista de Sistema
- Especificador de requisitos



Desarrolladores

- Arquitecto de Software
- Diseñador
- Diseñador de Interfaz de Usuario
- Diseñador de Cápsulas
- Diseñador de Bases de Datos
- Implementador
- Integrador



Gestores

- Jefe de Proyecto
- Jefe de Control de Cambios
- Jefe de Configuración
- Jefe de Pruebas
- Jefe de Despliegue
- Ingeniero de Procesos
- Revisor de Gestión del Proyecto
- Gestor de Pruebas



Apoyo

- Documentador Técnico
- Administrador de Sistema
- Especialista en herramientas
- Desarrollador de cursos
- Artista gráfico



Especialista en Pruebas

- Especialista en Pruebas (Tester)
- Analista de Pruebas
- Diseñador de Pruebas



Otros Roles

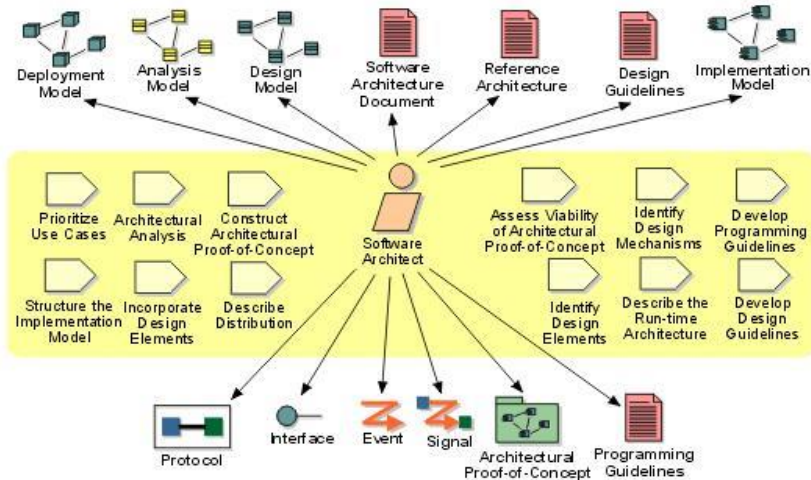
- Stakeholders
- Revisor
- Coordinador de revisiones
- Revisor Técnico.



Integrar equipos: negocio, desarrollo y operaciones



Roles en RUP



- **Modelo de Despliegue:** Muestra la configuración de nodos de proceso en tiempo de ejecución, los enlaces de comunicación entre ellos y las instancias de componentes y objetos que residen en ellos.
- **Modelo de Análisis:** Contiene los resultados del análisis de los casos de uso.

- **Modelo de Diseño:** Es un objeto de modelado que describe la realización de los Casos de Uso y sirve como una abstracción del modelo de implementación y su código fuente.

Contiene los siguientes Artefactos:

Protocolo: Es una especificación del comportamiento deseado que puede tener lugar a través de una conexión (una especificación explícita del acuerdo contractual entre los participantes en el protocolo).

Interfaz: Es un artefacto que define un conjunto de comportamientos, propuesto por un modelo clasificador.

Evento: Es la especificación de una ocurrencia en el espacio y tiempo; o en el buen español: un suceso o algo que a lo que el sistema debe responder.

Señal: Es una entidad de comunicación asíncrona que puede ocasionar un estado transitorio en la máquina de estados de un objeto que recibe dicha señal.

Cápsula: "Una cápsula es un patrón de diseño, lo que representa un encapsulado hilo de control en el sistema. Las cápsulas tienen puertos, a través de los cuales se comunican con otras cápsulas".

- **Documento de Arquitectura de Software:** Es un documento que provee una visión amplia arquitectónica del sistema, usando diferentes vistas arquitectónicas para representar diferentes aspectos del sistema.



Roles en RUP

- **Arquitectura de Referencia:** Es en esencia un patrón arquitectónico pre-definido o conjunto de patrones, posiblemente parcial o completamente instanciado, diseñado y probado para su uso en negocios particulares y contextos técnicos.
- **Directrices de Diseño:** Es un producto de la definición de arquitectura. Muchas personas necesitarán este documento, puesto que describe las directrices para ser seguidas durante el diseño e implementación.
- **Modelo de implementación:** El modelo de implantación es una colección de componentes y subsistemas de la aplicación que los contengan.
- **Prueba-de-Concepto arquitectónico:** Es una solución que simplemente puede ser conceptual, para los requerimientos de significado arquitectónico que son identificados en la concepción.
- **Directrices de Programación:** Describe los convenios a ser usados cuando se trabaje con el lenguaje de programación.





¡Gracias!

